Análisis técnico del acelerador criptográfico CUDA

Intel Core i7-1255U · NVIDIA GeForce RTX 2050 (GA107M, sm_86) · CUDA 12.4

---

1. El Problema Fundamental: SHA-256 es Secuencial por Diseño

SHA-256 aplica una **función de compresión iterativa de 64 rounds**. Cada round calcula un nuevo estado a partir del estado del round anterior. Esta **dependencia de datos en cadena** hace que sea imposible paralelizar los 64 rounds de un único hash SHA-256: el round N+1 no puede comenzar hasta que el round N haya terminado.

Pseudocódigo SHA-256

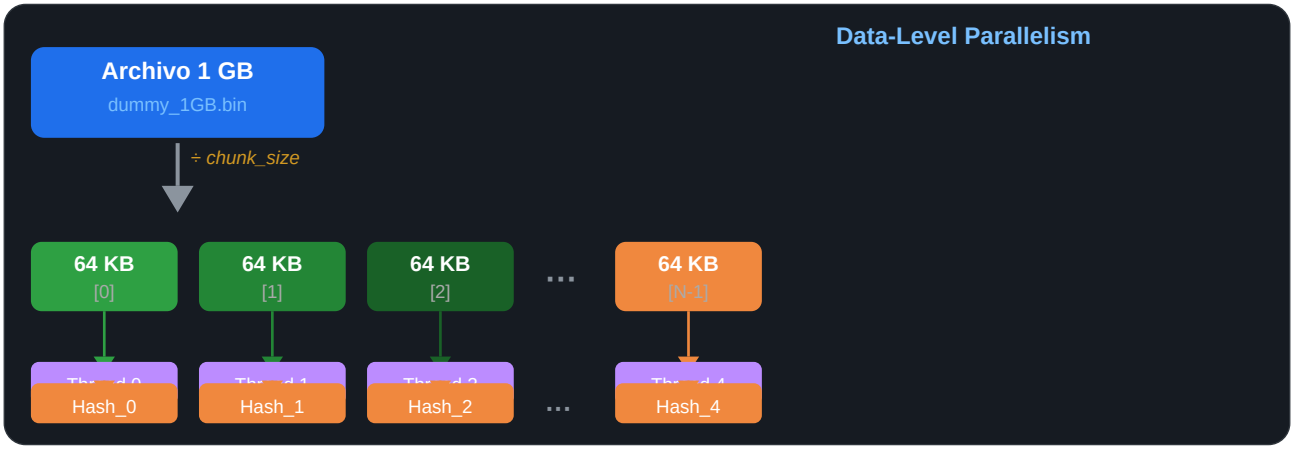
```
state = IV                                  // valores iniciales (constantes fijas)
for i in range(64):                         // cada round depende del anterior
    t1 = h + Σ■(e) + Ch(e,f,g) + K[i] + w[i]
    t2 = Σ■(a) + Maj(a,b,c)
    (a,b,c,d,e,f,g,h) = (t1+t2, a, b, c, d+t1, e, f, g)
state += (a, b, c, d, e, f, g, h)     // ← resultado = entrada del siguiente

→ Imposible paralelizar internamente: round N+1 requiere el estado del round N
```

---

2. La Solución: Paralelismo entre Hashes (Data-Level Parallelism)

En lugar de paralelizar *dentro* de un hash, paralelizamos *entre* hashes. El archivo de 1 GB se divide en miles de **chunks de tamaño fijo**. Cada chunk es completamente independiente de los demás — no comparte estado ni resultados intermedios. Esto permite que la GPU calcule **miles de hashes SHA-256 simultáneamente**, uno por thread CUDA, sin ningún tipo de sincronización ni comunicación entre hilos.

Figura 1: Mapeo de chunks a CUDA threads (Data-Level Parallelism)



***Nota:** Esta estrategia se llama **Data Parallelism** o paralelismo embarazosamente paralelo ('embarrassingly parallel'). Es la misma técnica usada en minería de criptomonedas, verificación de bloques en blockchain, y aceleración de HMAC-SHA256 en servidores TLS de alta demanda.*

---

3. Arquitectura CUDA de la RTX 2050 y Jerarquía de Ejecución

La NVIDIA GeForce RTX 2050 (GA107M) organiza su hardware en tres niveles anidados. El benchmark explota los tres niveles para maximizar el rendimiento:

Nivel Hardware	Unidad	Cantidad RTX 2050	Rol en el benchmark
GPU	SM (Streaming Multiprocessor)	16	Unidad de proceso independiente
SM	Warps (grupos de 32 threads)	48 máx / SM	Unidad de scheduling del SM
Warp	CUDA Threads	32	Ejecutan el mismo código
—	Registros por SM	65.536	Memoria privada ultrarrápida
—	Threads concurrentes máx	24.576	16 SMs × 1.536 threads/SM

3.1 Grid-Stride Loop: Saturar los 16 SMs en Todo Momento

El kernel v1 lanzaba exactamente un thread por chunk. Con 256KB/chunk → 4.096 threads. Con 64KB/chunk → 16.384 threads. El kernel v2 lanza un número fijo de threads igual a la capacidad máxima del hardware, y cada thread procesa múltiples chunks en un **bucle con paso igual al total de threads lanzados (grid-stride loop)**:

CUDA C — sha256_gpu_v2.cu (grid-stride kernel)

```
__global__ void sha256_kernel_v2(
    const uint8_t *d_in, uint8_t *d_out,
    int chunk_size, long num_chunks)
{
    // Grid-stride: cada thread avanza de stride en stride
    for (long idx = blockIdx.x * blockDim.x + threadIdx.x;
         idx < num_chunks;
         idx += gridDim.x * blockDim.x)    // ← stride = total threads
    {
        // Calcula SHA-256 del chunk idx en solitario, sin comunicación
        sha256_hash_chunk(d_in + idx*chunk_size, chunk_size,
                          d_out + idx*32);
    }
}
```

---

4. La Optimización Central: Reducir Registros y Maximizar Ocupancia

La **ocupancia** (SM occupancy) mide qué fracción de los warps disponibles por SM están activos. Más warps activos = mejor capacidad de ocultar latencias de memoria. El cuello de botella de v1 era la **presión de registros**: el message schedule estándar requiere un array w[64] de 256 bytes = 64 registros de 32 bits por thread. Con ~100 registros totales y 256 threads/bloque = 25.600 registros/bloque. La RTX 2050 tiene 65.536 registros/SM → solo 2 bloques activos/SM = 33% de ocupancia.

Figura 2: Ventana deslizante w[16] vs w[64] estándar y efecto en la ocupancia

Parámetro	v1 (Baseline)	v2 (Optimizado)	Delta
Array message schedule	w[64] — 256 bytes	w[16] — 64 bytes	–192 bytes
Registros totales/thread	~100	≤48 (maxrregcount)	–52 registros
Bloques activos/SM	2	5	+3 bloques
Threads activos/SM	512 / 1.536	1.280 / 1.536	+768 threads
Ocupancia estimada	33.3%	83.3%	+50 puntos %
Throughput kernel 64 KB	**2.81 GB/s**	**16.81 GB/s**	**+499% ■**

4.1 La Fórmula de la Ventana Deslizante

La expansión del message schedule SHA-256 sigue la recurrencia:

Message Schedule — Comparativa v1 vs v2

```
// Estándar (v1): w[64] — necesita todas las 64 palabras en memoria
for i in range(16, 64):
    w[i] = σ■(w[i-2]) + w[i-7] + σ■(w[i-15]) + w[i-16]

// Ventana deslizante (v2): w[16] circular — sobreescribe en-lugar
for i in range(16, 64):
    j = i % 16            // índice circular (0..15)
    // w[j] antes del assignment == w[i-16]  (ya no se necesitará)
    w[j] = σ■(w[(i-2)%16]) + w[(i-7)%16] + σ■(w[(i-15)%16]) + w[j]
    // w[j] después del assignment == w[i]  ← correcto para este round
```

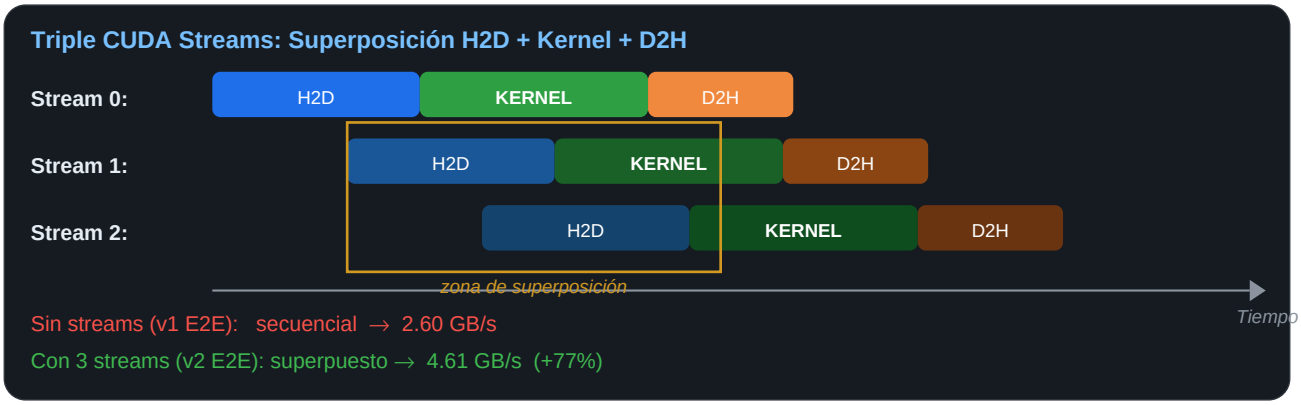
---

5. El Cuello de Botella PCIe y los Triple CUDA Streams

Aunque el kernel CUDA v2 alcanza **16.81 GB/s** de throughput criptográfico puro, el rendimiento End-to-End real del sistema es de **4.61 GB/s**. La diferencia se debe al bus **PCIe 3.0 ×16** que conecta la CPU (RAM) con la GPU (VRAM): el archivo de 1 GB debe transferirse antes de procesarse, y los 32 bytes de hash de cada chunk deben copiarse de vuelta. En la práctica, el ancho de banda efectivo de PCIe para este workload es de ~10 GB/s — un factor 11× menor que la memoria GDDR6 de la GPU.

```
Análisis de rendimiento

  Throughput kernel puro  = 16.81 GB/s  (computación dentro de la VRAM)
  Throughput E2E real     =  4.61 GB/s  (incluye H2D + kernel + D2H)
  Eficiencia PCIe         =  4.61 / 16.81 = 27.4%

  → El 72.6% del tiempo total se consume en transferencia de datos,
    no en el cálculo criptográfico. El PCIe es el cuello de botella.
```

Figura 3: Triple CUDA Streams — Superposición de Etapas



La RTX 2050 dispone de motores DMA independientes para transferencias H2D y D2H. Con 3 streams sobre 3 segmentos del archivo, se crea un pipeline donde: mientras el **kernel** procesa el segmento N, el **motor H2D** ya carga el segmento N+1 y el **motor D2H** descarga los hashes del segmento N-1 — todo en paralelo. Esto redujo el tiempo E2E en un +77% respecto a v1 (2.60 → 4.61 GB/s).

6. Resultados Completos del Benchmark

Figura 4: Throughput comparativo — todas las plataformas y configuraciones

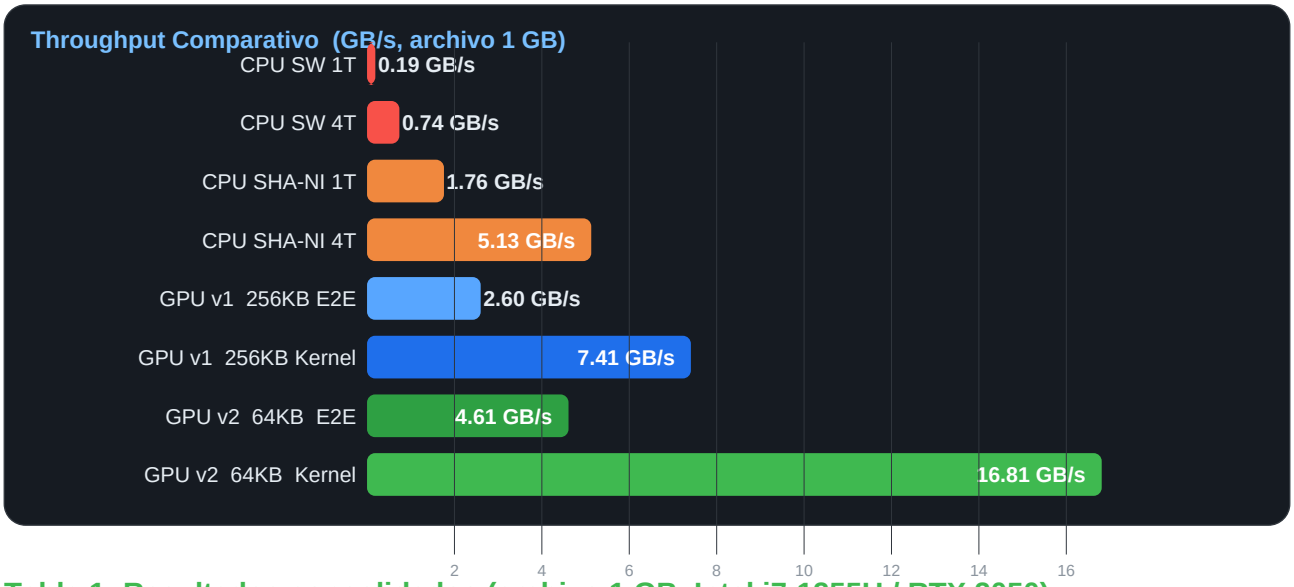


Tabla 1: Resultados consolidados (archivo 1 GB, Intel i7-1255U / RTX 2050)

Plataforma	Config.	Throughput	Speedup vs SW 1T	Energía (J/GB)
	1 Thread	189.7 MB/s	1.00×	69.76
CPU SW (C puro)	2 Threads	372.2 MB/s	1.96×	44.23
	4 Threads	735.7 MB/s	3.88×	32.89
	1 Thread	1.763 MB/s	9.29×	7.00
CPU SHA-NI	2 Threads	3.315 MB/s	17.47×	5.37
	4 Threads	5.126 MB/s	27.02×	5.17
	4 KB	1.966 MB/s	10.4×	4.74
GPU v1 (E2E)	256 KB	2.662 MB/s	14.0×	3.77
	256 KB	7.588 MB/s	40.0×	1.32
GPU v2 (E2E)	**64 KB**	**4.717 MB/s**	**24.9×**	**2.16**
GPU v2 (Kernel)	**64 KB**	**16.81 GB/s**	**88.6×**	**—**
GPU v2 (Kernel)	**4 KB**	**11.57 GB/s**	**61.0×**	**1.90 ★**

7. Las 8 Optimizaciones del Kernel v2

1. __constant__ K[64]: Los 64 valores de la tabla K de SHA-256 se ubican en memoria constante de CUDA. Cuando todos los threads de un warp acceden al mismo valor K[i], el hardware emite una única transacción de memoria en lugar de 32 lecturas individuales (broadcast).

2. Ventana deslizante w[16]: El array del message schedule pasa de 64 a 16 palabras mediante escritura circular in-place. Ahorra 48 registros/thread, triplicando los warps activos por SM.

3. #pragma unroll 64 rounds: El compilador emite los 64 rounds como código lineal (sin bucle). Elimina el overhead de saltos condicionales y permite al planificador de instrucciones maximizar el ILP.

4. __byte_perm() para carga big-endian: La instrucción PTX 'sel.b32' convierte el orden de bytes de little-endian (x86) a big-endian (SHA-256) en un único ciclo de reloj, reemplazando 4 shifts + 3 ORs.

5. Padding block inline: El bloque de padding SHA-256 (requerido por la especificación) se construye directamente como 16 valores uint32 en registros, sin array local en memoria.

6. Grid-stride loop: En lugar de un thread por chunk, se lanza un grid fijo que satura todos los SMs. Cada thread procesa múltiples chunks en un bucle strideado, eliminando el overhead de launches adicionales y garantizando 100% de actividad de los 16 SMs.

7. Memoria pinned (cudaMallocHost): La memoria del host se reserva como page-locked (no paginable). Esto permite al motor DMA de la GPU acceder directamente a ella sin copia intermedia del SO, duplicando el ancho de banda efectivo en las transferencias PCIe asíncronas.

8. Triple CUDA Streams: El archivo se divide en 3 segmentos. Cada segmento se procesa en un stream independiente, solapando H2D del segmento N+1 con el kernel del segmento N y el D2H del segmento N-1 en los motores DMA separados de la GPU.

---

8. Conclusiones

1. **Tipo de paralelismo:** SHA-256 se paraleliza a nivel de datos (DLP), no a nivel de instrucción. Cada thread calcula un hash independiente; no hay comunicación entre threads ni sincronización de estado.

2. **La ocupancia es la métrica crítica:** El factor de mejora más grande (33%→83%) se obtuvo únicamente reduciendo los registros de 100 a 48 por thread. Esto ilustra que en kernels compute-bound, la presión de registros domina sobre la frecuencia de reloj o el número de SMs.

3. **El cuello de botella real es el PCIe:** La GPU logra 16.81 GB/s de cómputo pero solo 4.61 GB/s E2E. Para cargas de trabajo donde los datos residen en RAM de host, el PCIe es el límite sistémico — no la capacidad de cómputo de la GPU.

4. **Eficiencia energética:** GPU v2 logra 1.90 J/GB — 36.7× mejor que el software de CPU y 2.7× mejor que SHA-NI en 4 cores, con 10.7 W de consumo promedio.

5. **Chunk size óptimo cambia con la arquitectura:** v1 prefería 256 KB (más trabajo por thread compensaba la baja ocupancia); v2 prefiere 64 KB (más paralelismo ya que la alta ocupancia no requiere compensación por threads escasos).